

REPORT — ESERCIZIO DI CRITTOGRAFIA

Data: 23/04/2026 | Studente: Lorenzo Ventanni

0. Testo dell'Esercizio:

Esercizio di oggi: Crittografia.

Dato un messaggio cifrato cercare di trovare il testo in chiaro:

Messaggio cifrato: "HSNFRGH"

Secondo esercizio

QWJhIHZ6b2VidHI2bmdyIHb1ciB6ciBhciBuchBiZXRi

Buon divertimento

Screenshot del testo originale dell'esercizio.

1. Esercizio 1 — Cifrario di Cesare:

Messaggio cifrato:

HSNFRGH

Approccio risolutivo:

Ho risolto questo esercizio in modo **logico-analitico**, senza ausilio di strumenti esterni. Esaminando la struttura del messaggio cifrato, ho ipotizzato l'utilizzo di un **Cifrario di Cesare** con uno shift di **+3 posizioni** nell'alfabeto. Applicando la decifrazione inversa (shift -3) a ciascuna lettera, ho ottenuto il testo in chiaro.

Dettaglio della decifrazione (shift -3):

Lettera cifrata	H	S	N	F	R	G	H
Shift -3 applicato	H → E	S → P	N → I	F → C	R → O	G → D	H → E
Lettera in chiaro	E	P	I	C	O	D	E

Risultato:

EPICODE

2. Esercizio 2 — Doppia Cifratura (Base64 + ROT13):

Messaggio cifrato:

QWJhIHZ6b2VidHI2bmdyIHB1ciB6ciBhciBucHBldGI

Difficoltà incontrate:

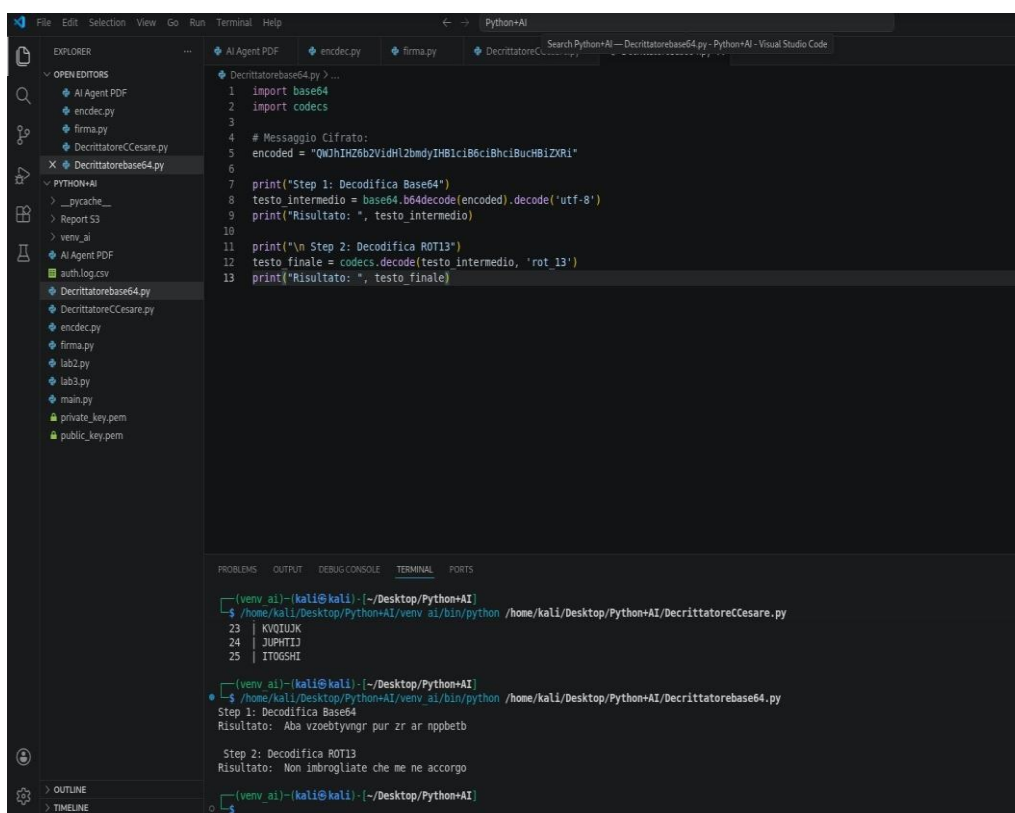
A differenza del primo esercizio, in questo caso ho incontrato **difficoltà nel riconoscere il tipo di cifratura utilizzata**. La stringa non presentava caratteristiche immediatamente riconducibili a un singolo cifrario a me attualmente noto. Non avendo identificato autonomamente la tecnica, ho deciso di richiedere un **aiuto ai colleghi e a Claude**, che mi hanno guidato nella comprensione del metodo adottato.

Approccio risolutivo (con supporto):

Grazie al supporto ricevuto, ho scoperto che il messaggio era stato cifrato con una **doppia codifica**: prima **Base64**, poi **ROT13**. Ho quindi implementato uno script Python (visibile nello screenshot allegato) che eseguisse i due passi di decodifica in sequenza.

Step	Operazione	Risultato intermedio
1	Decodifica Base64	Aba vzoebtyvngr pur zr ne nppbetb
2	Decodifica ROT13	Non imbrogliate che me ne accorgo

Screenshot del codice Python:



```
1 import base64
2 import codecs
3
4 # Messaggio Cifrato:
5 encoded = "QWJhIHZ6b2VidHI2bmdyIHB1ciB6ciBhciBucHBldGI"
6
7 print("Step 1: Decodifica Base64")
8 testo_intermedio = base64.b64decode(encoded).decode('utf-8')
9 print("Risultato: ", testo_intermedio)
10
11 print("\n Step 2: Decodifica ROT13")
12 testo_finale = codecs.decode(testo_intermedio, 'rot_13')
13 print("Risultato: ", testo_finale)
```

```
(venv ai)-(kali@kali): ~/Desktop/Python+AI
$ /home/kali/Desktop/Python+AI/venv_ai/bin/python /home/kali/Desktop/Python+AI/DecrittatoreCesare.py
23 | KVQIUJK
24 | JUPHTIJ
25 | ITOGSHI

(venv ai)-(kali@kali): ~/Desktop/Python+AI
$ /home/kali/Desktop/Python+AI/venv_ai/bin/python /home/kali/Desktop/Python+AI/DecrittatoreBase64.py
Step 1: Decodifica Base64
Risultato: Aba vzoebtyvngr pur zr ne nppbetb

Step 2: Decodifica ROT13
Risultato: Non imbrogliate che me ne accorgo

(venv ai)-(kali@kali): ~/Desktop/Python+AI
```

Script Python per la decodifica Base64 + ROT13 — eseguito in VS Code su Kali Linux.

Risultato finale:

Non imbrogliate che me ne accorgo

3. Conclusioni:

L'esercizio è stato utile per approfondire le tecniche di cifratura classica.

Il primo messaggio è stato decifrato autonomamente tramite analisi logica del Cifrario di Cesare. Il secondo ha richiesto un supporto esterno e un lavoro collaborativo.

Entrambi gli esercizi hanno contribuito a consolidare le competenze di base in ambito crittografico e nella scrittura di script Python.